ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH

MÔN HỌC: HỆ ĐIỀU HÀNH (CS333)

LAB 2
SYSTEM CALLS

Sinh viên thực hiện:	Võ Nhất Bảo	24120166
	Phạm Lê Huy Hoàng	24120182
Mã nhóm:	24120166_24120182	

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Thông tin nhóm và phân công công việc 2

2 Tổng quan và môi trường thực hiện 2

3 Task 3.1 – Using gdb 2

4 Task 3.2 – Sandbox a command 4

5 Task 3.3 – Sandbox with allowed pathnames 6

6 Task 3.4 – Attack xv6 6

7 Kiểm thử 7

8 Thảo luận và tự đánh giá 7

9 Tài liệu tham khảo 7

1. Thông tin nhóm và phân công công việc

STT	MSSV	Họ và tên	Công việc đảm nhận	Đóng góp
1	24120166	Võ Nhất Bảo	Task 3.1: chạy gdb với QEMU, trả lời sáu câu hỏi (`answers-syscall.txt`), thu thập ảnh minh chứng. Task 3.2: system call `interpose`, chặn system call theo mask, kế thừa mask qua `fork`. Viết các phần báo cáo tương ứng.	50%
2	24120182	Phạm Lê Huy Hoàng	Task 3.3: cho phép `open/exec` theo pathname. Task 3.4: tấn công xv6 (`user/attack.c`). Viết các phần báo cáo tương ứng.	50%
Công việc chung			Chạy autograder, rà soát chéo mã nguồn, `time.txt`, hoàn thiện báo cáo, đóng gói bài nộp.	Cả hai

Bảng 1. Phân công công việc và tỷ lệ đóng góp của các thành viên.

Task	Nội dung	Tệp chính	Hoàn thành	Autograder
3.1	Using gdb	`answers-syscall.txt`	100%	5/5
3.2	Sandbox theo mask	`kernel/{syscall,sysproc,proc}.c`, `kernel/proc.h`	100%	20/20
3.3	Sandbox theo pathname	*chờ bổ sung*	*chờ bổ sung*	*chờ bổ sung* /20
3.4	Attack xv6	`user/attack.c`	*chờ bổ sung*	*chờ bổ sung* /14
–	Số giờ thực hiện	`time.txt`	*chờ bổ sung*	*chờ bổ sung* /1
Tổng kết quả `make grade`				*chờ bổ sung* /60

Bảng 2. Mức độ hoàn thành các yêu cầu của Lab 2. Các test thuộc từng task được liệt kê ở Bảng 6.

2. Tổng quan và môi trường thực hiện

Lab 2 đi vào bên trong kernel xv6: dùng GDB để theo dõi một system call và lần ra nguyên nhân của một kernel panic (Task 3.1), thêm system call mới `interpose` để "sandbox" một process (Task 3.2, 3.3), và khai thác một lỗi rò rỉ bộ nhớ được cố ý đưa vào kernel (Task 3.4).

Môi trường. Giống Lab 1: NixOS 26.05 (x86-64), công cụ khai báo trong `shell.nix` gồm `riscv64-none-elf-gcc` 15.2.0, GNU Binutils 2.46, QEMU 10.2.4 (3 CPU), GDB 17.2, GNU Make 4.4.1 và Python 3.13 (autograder). Mã nguồn: `git://g.csail.mit.edu/xv6-labs-2026`, nhánh `syscall`, commit gốc `99161db`; `conf/lab.mk` đặt `LAB=syscall`.

Quy trình. Nhánh `syscall` được tạo từ `origin/syscall` (`git checkout -b syscall --track origin/syscall`). Mỗi task được build và chạy thử bằng `make qemu`, chấm riêng bằng `./grade-lab-syscall <tên test>` rồi commit; hai tệp chấm điểm `grade-lab-syscall` và `gradelib.py` được giữ nguyên.

Lỗi cố ý của lab. Makefile biên dịch kernel với `-DLAB_SYSCALL`. Khi macro này được định nghĩa, `uvmalloc()` bỏ lệnh `memset(mem, 0, sz)` (`kernel/vm.c`, dòng 268–270), còn `kfree()`/`kalloc()` bỏ việc ghi rác lên trang (`kernel/kalloc.c`, dòng 57–60 và 85–88). Hệ quả: trang nhớ cấp cho process mới vẫn giữ nội dung của process trước. Đây là cơ sở của Task 3.4, và cũng ảnh hưởng tới kiểm thử thủ công của Task 3.2 (mục 8).

3. Task 3.1 – Using gdb

Mở hai terminal: `make qemu-gdb` khởi động QEMU với `-S -gdb tcp::26000`, nên CPU dừng ngay từ lệnh đầu tiên và chờ debugger; `gdb` đọc `.gdbinit` để kết nối (`target remote`) và nạp symbol từ `kernel/kernel`. Mọi số liệu dưới đây lấy từ phiên chạy thật trên nhánh `syscall` *trước* khi sửa kernel; địa chỉ phụ thuộc bản build. Ảnh chụp do script `scripts/capture_lab2.py` tạo: script gõ lệnh vào gdb thật trong xterm rồi chụp cửa sổ. Câu trả lời đầy đủ (tiếng Anh) nằm trong `answers-syscall.txt`.

```
(gdb) b syscall
Breakpoint 1 at 0x80001d6a: file kernel/syscall.c, line 138.
(gdb) c
Continuing.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:138
138     {
(gdb) backtrace
#0  syscall () at kernel/syscall.c:138
#1  0x0000000080001b2a in usertrap () at kernel/trap.c:68
#2  0x0000003ffffff09c in ?? ()
(gdb) n
140       struct proc *p = myproc();
(gdb) n
142       num = p->trapframe->a7;
(gdb) p /x *p
$1 = {lock = {locked = 0x0, name = 0x80007130, cpu = 0x0}, state = 0x4, chan = 0x0, killed = 0x0, xstate = 0x0,
  pid = 0x1, parent = 0x0, kstack = 0x3fffffd000, sz = 0x4000, pagetable = 0x87f52000, trapframe = 0x87f56000,
  context = {ra = 0x8000135c, sp = 0x3fffffdc60, s0 = 0x3fffffdc90, s1 = 0x80007cf0, s2 = 0x800078c0, s3 = 0x0,
    s4 = 0x3, s5 = 0x80018990, s6 = 0x0, s7 = 0x18c, s8 = 0xffffffffffffffff, s9 = 0x400, s10 = 0x2, s11 = 0x38},
  ofile = {0x0 <repeats 16 times>}, cwd = 0x80015e00, name = {0x69, 0x6e, 0x69, 0x74, 0x0 <repeats 12 times>}}
(gdb) p p->trapframe->a7
$2 = 15
(gdb) p /x $sstatus
$3 = 0x200000022
(gdb)
```

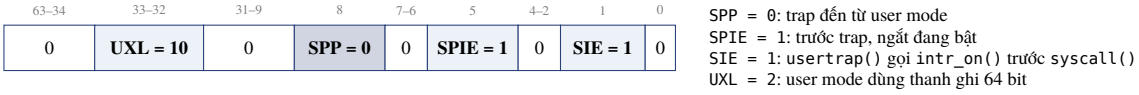
Hình 1. Breakpoint tại `syscall`: `backtrace`, nội dung `struct proc`, `trapframe->a7` và `sstatus` (Q1–Q3).

Q1. Hàm nào đã gọi `syscall`? Theo frame `#1` của `backtrace`, đó là `usertrap()` tại `kernel/trap.c:68`. Frame `#2` ở địa chỉ `0x3ffffff09c` = `TRAMPOLINE + 0x9c`, chính là nhãn `userret`: `uservec` gọi `usertrap()` bằng lệnh `jalr t0`, nên địa chỉ trả về là lệnh ngay sau nó. GDB hiện `??` vì mã trampoline được link ở `0x80006000` (symbol `uservec`), còn khi chạy nó được map ở địa chỉ ảo `TRAMPOLINE = MAXVA - PGSIZE = 0x3ffffff000`.

Q2. Giá trị của `p->trapframe->a7` là gì và nó biểu diễn điều gì? `a7 = 15`, tức `SYS_open` trong `kernel/syscall.h`. Stub trong `user/usys.S` nạp số hiệu system call vào `a7` rồi gọi `ecall`; `uservec` lưu các thanh ghi user vào trapframe, và `syscall()` đọc lại `a7` từ đó để chọn hàm xử lý.

Gợi ý của đề (xem `user/initcode.S`, nơi giá trị là `SYS_exec`) dựa trên bản xv6 cũ. Bản 2026 không còn tệp này: `forkret()` (`kernel/proc.c`) gọi trực tiếp `kexec("/init")` trong kernel cho process đầu tiên. Vì vậy system call đầu tiên đi qua `syscall()` là `open("console", O_RDWR)` ở đầu `main` của `user/init.c`. Kết quả `p /x *p` khẳng định điều này: `pid = 0x1`, `name = {0x69, 0x6e, 0x69, 0x74}` = `"init"`, `state = 0x4` (`RUNNING`).

Q3. CPU đang ở mode nào trước đó? User mode. `p /x $sstatus` cho `0x200000022`. Bit `SPP` (bit 8) lưu mode trước khi trap: 0 là user, 1 là supervisor. Các bit khác cũng khớp với đường đi của system call (Hình 2).

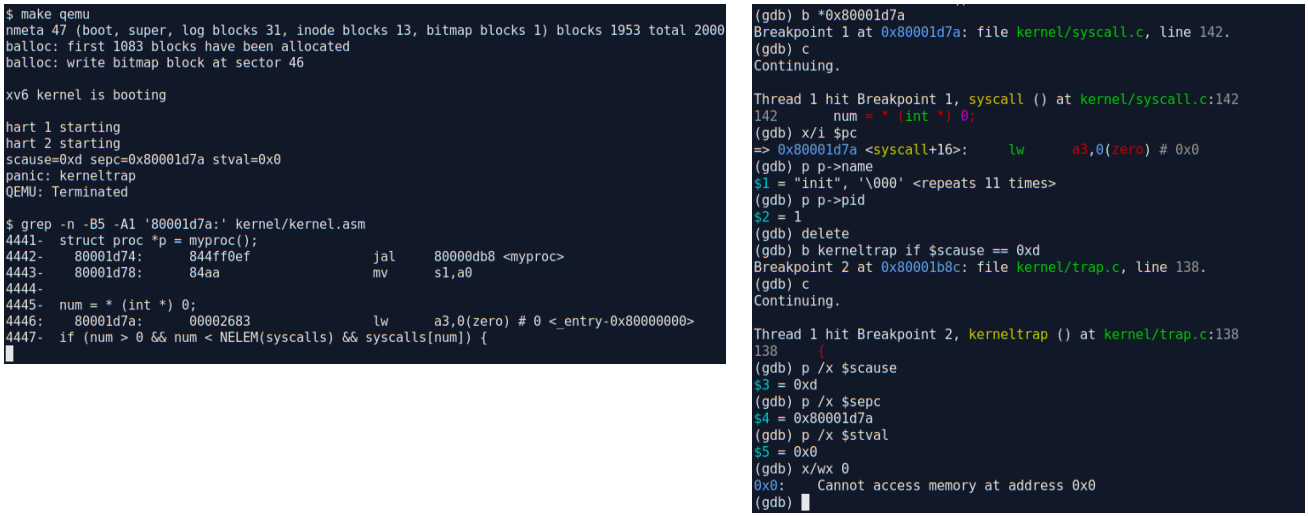


Hình 2. Giải mã `sstatus = 0x200000022`.

Q4. Lệnh assembly nào gây panic? Thanh ghi nào ứng với biến `num`? Thay `num = p->trapframe->a7;` bằng `num = *(int *) 0;` rồi chạy `make qemu`, kernel in `scause=0xd sepc=0x80001d7a stval=0x0` và `panic: kerneltrap`. Tìm `80001d7a` trong `kernel/kernel.asm` (Hình 3, trái) được lệnh

```
80001d7a:  00002683    lw  a3,0(zero)
```

tức nạp 4 byte từ địa chỉ `zero + 0` = `0` vào thanh ghi `a3`. Vậy `num` nằm trong **`a3`**; lệnh kế tiếp `addiw a4,a3,-1` dùng `a3` để kiểm tra `num > 0 && num < NELEM(syscalls)`. Trong gdb, breakpoint `b *0x80001d7a` dừng đúng tại dòng 142 của `syscall.c` và `x/i $pc` hiện cùng lệnh (Hình 3, phải).



Hình 3. Trái: panic khi đọc địa chỉ 0 và lệnh tương ứng trong `kernel.asm`. Phải: gdb tại lệnh lỗi và tại `kerneltrap` (Q4–Q6).

Q5. Vì sao kernel bị crash?

Kernel xv6 chạy với phân trang bật, dùng kernel page table do `kvmmake()` (`kernel/vm.c`) tạo. Bảng này chỉ map các vùng ở bảng bên, khớp Hình 3-3 của sách xv6; địa chỉ ảo `0` không thuộc vùng nào. Khi lệnh `lw` đọc địa chỉ 0, MMU không tìm được PTE hợp lệ và sinh exception:

- `scause = 0xd` = 13: *load page fault* (bit ngắt = 0, mã exception 13 theo đặc tả RISC-V privileged);
- `stval = 0x0`: địa chỉ ảo gây lỗi;
- `sepc = 0x80001d7a`: lệnh `lw` gây lỗi.

Trap đi vào `kernelvec` rồi `kerneltrap()`. Đây không phải ngắt thiết bị và kernel không bao giờ được phép page fault, nên `kerneltrap()` in ba thanh ghi trên rồi gọi `panic("kerneltrap")`. GDB xác nhận bằng breakpoint có điều kiện `b kerneltrap if $scause == 0xd`: `$scause = 0xd`, `$sepc = 0x80001d7a`, `$stval = 0x0`. Lệnh `x/wx 0` cũng không đọc được địa chỉ 0.

Vùng	Địa chỉ ảo	Quyền
PLIC	`0x0c000000–0x10000000`	RW
UART0	`0x10000000` (1 trang)	RW
VIRTIO0	`0x10001000` (1 trang)	RW
Mã kernel	`0x80000000–etext`	RX
Dữ liệu + RAM	`etext–0x88000000`	RW
Kernel stack	dưới `TRAMPOLINE`	RW
Trampoline	`0x3ffffff000`	RX

Bảng 3. Các vùng được map trong kernel page table; `0x0` nằm ngoài tất cả.

> Cùng địa chỉ 0 lại *hợp lệ* trong user page table, vì mã của chương trình user bắt đầu ở địa chỉ ảo 0. Kernel page table không map bộ nhớ user, nên kernel phải đọc dữ liệu user qua `copyin/fetchaddr`.

Q6. Process nào đang chạy khi kernel panic? PID là bao nhiêu? Tại breakpoint `0x80001d7a`: `p p->name` = `"init"`, `p p->pid` = 1. Đó là process **`init`** (PID 1), đang thực hiện system call đầu tiên, nên shell chưa kịp khởi động. Sau khi trả lời, câu lệnh `num = p->trapframe->a7;` được khôi phục (script tự khôi phục và `git status` sạch).

4. Task 3.2 – Sandbox a command

Yêu cầu. Thêm system call `int interpose(int mask, const char *path)`. Nếu bit `n` của `mask` được bật thì system call số `n` của process gọi *không được thực thi* và trả về -1. Process con tạo bằng `fork` thừa hưởng hạn chế của cha. Ở task này `path` luôn là `"-"` (được dùng ở Task 3.3).

4.1. Các tệp đã sửa và trường mới trong `struct proc`

Tệp	Thay đổi
`Makefile`	Thêm `$U/_sandbox`, `$U/_sbtest` vào `UPROGS` (khối `ifeq ($(LAB),syscall)`).
`kernel/syscall.h`	`#define SYS_interpose 23`.
`user/usys.pl`, `user/user.h`	`entry("interpose")` sinh stub trong `user/usys.S`; khai báo `int interpose(int, const char *)`.
`kernel/syscall.c`	Khai báo `sys_interpose`, thêm `[SYS_interpose]` vào bảng `syscalls[]`; hàm `sandbox_rejects()` và phép kiểm tra trong `syscall()`.
`kernel/sysproc.c`	Hàm xử lý `sys_interpose()`.
`kernel/proc.h`, `kernel/proc.c`	Trường `sbmask`; `kfork()` sao chép cho con; `freeproc()` đặt lại 0.

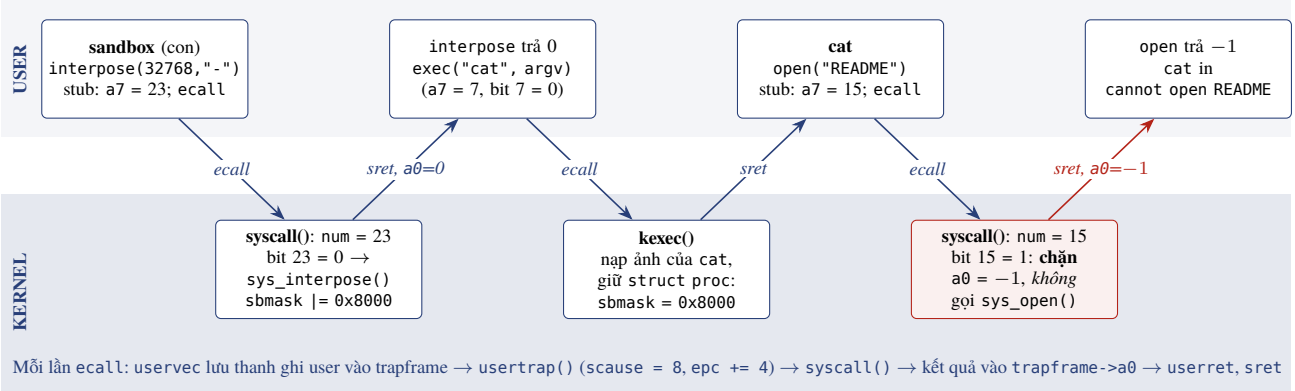
Bảng 4. Các tệp được sửa trong Task 3.2.

Trường	Kiểu	Ý nghĩa	Vòng đời
`sbmask`	`int`	Bit `n` = 1 thì system call số `n` bị từ chối.	Ghi bởi `sys_interpose()` (chỉ OR thêm bit); `kfork()` sao chép; `kexec()` giữ nguyên; `freeproc()` đặt 0.
chờ bổ sung		Pathname được phép (Task 3.3).	

Bảng 5. Các trường mới trong `struct proc` (`kernel/proc.h`).

4.2. Đường đi của lời gọi `interpose` và của một system call bị chặn

Hình 4 theo dõi lệnh `sandbox 32768 - cat README`. `sandbox` fork một process con; process con gọi `interpose(32768, "-")` rồi `exec` chương trình `cat`. $32768 = 2^{15}$ = `1 << SYS_open`.



Hình 4. `interpose` ghi mask vào `struct proc`; mask sống qua `exec` và chặn `open` của `cat` ngay tại dispatcher.

4.3. Thiết kế và cài đặt

- **Kiểm tra một lần, trước khi gọi handler.** Phép kiểm tra nằm trong `syscall()`, sau khi `num` đã được xác nhận nằm trong bảng ($1 \le$ `num` ≤ 23, nên phép dịch `sbmask >> num` luôn hợp lệ với `int` 32 bit). System call bị chặn không chạy handler nên không có tác dụng phụ: `fork` bị chặn không tạo process con. Cách này không phải sửa từng handler, và chặn được cả system call không có đối số như `getpid`, `fork` (test `sandbox_block`).
- **Hạn chế chỉ được cộng dồn.** `sys_interpose()` dùng `|=` chứ không gán. Test `sandbox_escape` chạy `sbtest escape` dưới mask `open`; chương trình gọi `interpose(0, 0)` và `interpose(0, "README")` để thử thoát. Nếu dùng phép gán, mask 0 xóa luôn bit 15 và process tự thoát sandbox.
- **Kế thừa qua `fork`, giữ qua `exec`.** `kfork()` sao chép `sbmask` cho process con. `kexec()` thay ảnh bộ nhớ nhưng giữ `struct proc`, nên mask đặt trước `exec` vẫn còn hiệu lực. Nhờ vậy `sandbox 32768 - sh < exec.sh` chặn được cả `ls` do `sh` fork ra (test `sandbox_fork`).
- **Khởi tạo đúng cho process mới.** `freeproc()` đặt `sbmask = 0` cùng chỗ với `pid`, `killed`, `xstate`. `allocproc()` chỉ cấp slot `UNUSED`, tức slot đã qua `freeproc()` hoặc mảng `proc[]` tĩnh vốn bằng 0, nên process không tạo bằng fork không thể nhận mask cũ.
- **Đồng bộ.** `sbmask` chỉ được process tự đọc/ghi (trong `syscall()`, `sys_interpose()`), được `kfork()` ghi trước khi process con thành `RUNNABLE`, và được `freeproc()` ghi khi process đã kết thúc. Vì vậy trường này thuộc nhóm "private to the process" như `sz`, `pagetable`, không cần giữ `p->lock`.

kernel/syscall.c – kiểm tra sandbox tại dispatcher

```
138 // Does the sandbox set up by interpose() reject system call num for p?
139 static int
140 sandbox_rejects(struct proc *p, int num)
141 {
142   return (p->sbmask >> num) & 1;
143 }
```

```
151   num = p->trapframe->a7;
152   if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
153     if (sandbox_rejects(p, num)) {
154       // Fail without running the handler.
155       p->trapframe->a0 = -1;
156     } else {
157       // Use num to lookup the system call function for num, call it,
158       // and store its return value in p->trapframe->a0
159       p->trapframe->a0 = syscalls[num]();
160     }
```

kernel/sysproc.c – sys_interpose

```
117 uint64
118 sys_interpose(void)
119 {
120   int mask;
121
122   argint(0, &mask);
123   myproc()->sbmask |= mask;
124   return 0;
125 }
```

kernel/proc.c – kfork() và freeproc()

```
293   // the child stays in the parent's sandbox.
294   np->sbmask = p->sbmask;
```

```
170   p->sbmask = 0;
```

`argint(0, &mask)` đọc đối số thứ nhất từ `trapframe->a0`; mask mới được OR vào mask cũ. Đối số `path` (`a1`) chưa được đọc ở task này.

4.4. Chạy thử

```
init: starting sh
$ sandbox 32768 - sh < exec.sh
$ "hello world!"
$ ls: cannot open .
$ $ echo SCRIPT_DONE
SCRIPT_DONE
$ sandbox 32768 - cat README
cat: cannot open README
$ sandbox 32768 - echo "hello"
"hello"
$ sandbox 128 - grep xv6 README
sandbox: exec grep failed
$ sandbox 32768 - sbtest getpid
sbtest: getpid ret 15
sbtest: alive
$ sandbox 2048 - sbtest getpid
sbtest: getpid ret -1
sbtest: alive
$ sandbox 2 - sbtest fork
sbtest: fork ret -1
sbtest: alive
$ sandbox 32768 - sbtest escape
sbtest: escape confined
sbtest: alive
$ sbtest getpid
sbtest: getpid ret 22
sbtest: alive
$
```

Hình 5. Các mask khác nhau trong xv6.

Mask	Bit	Kết quả
32768	`open` (15)	`sh < exec.sh`: `echo` vẫn in `"hello world!"`, `ls` báo `cannot open .`; `cat`: `cannot open README`; `echo` không mở tệp nên chạy bình thường.
128	`exec` (7)	`sandbox: exec grep failed`: chính process con của `sandbox` không exec được.
32768	`open` (15)	`sbtest getpid` trả PID thật (15): system call không bị mask vẫn chạy.
2048	`getpid` (11)	`getpid ret -1`, process vẫn `alive`.
2	`fork` (1)	`fork ret -1`, không có dòng `CHILD RAN`: handler không chạy.
32768	`open` (15)	`sbtest escape`: `escape confined`.
–	–	`sbtest getpid` chạy sau cùng, ngoài sandbox, trả PID 22: slot process tái sử dụng không giữ mask cũ.

5. Task 3.3 – Sandbox with allowed pathnames

CHỜ BỔ SUNG (Phạm Lê Huy Hoàng). Theo mục 5.2 của đề: trường pathname mới trong `struct proc` (bổ sung vào bảng trường ở mục 4.1), cách lấy `path` bằng `argstr` vào buffer `MAXPATH`, xử lý `"-"`, phép so sánh pathname của `open/exec` (gợi ý: mở rộng `sandbox_rejects()`), kế thừa qua `fork`, danh sách tệp đã sửa và đường đi của một lời gọi `open/exec` được phép. Lưu ý `sbtest escape` gọi lại `interpose(0, "README")`, và địa chỉ 0 hợp lệ trong user space nên `interpose(0, 0)` không làm `argstr` thất bại.

6. Task 3.4 – Attack xv6

CHỜ BỔ SUNG (Phạm Lê Huy Hoàng). Giải thích vì sao tấn công thành công: trang vật lý nào của `secret` được `kalloc()` cấp lại cho `attack`, secret nằm ở offset nào và cách xác định offset đó (đề trừ điểm nếu chỉ thử sai mà không giải thích).

7. Kiểm thử

Task	Test của autograder	Nội dung	Kết quả
3.1	`answers-syscall.txt`	Tệp trả lời tồn tại, đủ nội dung (chấm tay)	OK 5/5
3.2	`sandbox_mask`	Mask `open`, `exec`, cả hai; `exec` bị chặn	OK 5/5
3.2	`sandbox_fork`	Mask được `sh` và các process con thừa hưởng	OK 5/5
3.2	`sandbox_block`	`getpid`, `fork` bị chặn trả -1, không chạy handler	OK 5/5
3.2	`sandbox_escape`	Gọi lại `interpose` không gỡ được hạn chế	OK 5/5
3.3	`sandbox_{path,most,minus,forkpath}`	Pathname được phép cho `open`/`exec`	*chờ bổ sung*
3.4	`attack`	In ra secret của lần chạy `secret` trước	*chờ bổ sung*
–	`time`	`time.txt` chứa một số nguyên	*chờ bổ sung*

Bảng 6. Các test của `grade-lab-syscall`. Kết quả hiện tại của Task 3.1, 3.2 lấy từ `./grade-lab-syscall` trên nhánh `syscall`.

Mã của Task 3.2 biên dịch với `-Wall -Werror` không có cảnh báo; cảnh báo `RWX` của linker thuộc kernel gốc. Ngoài autograder, ảnh chạy thử ở Hình 5 kiểm tra thêm hai điều autograder không đo: system call không bị mask vẫn hoạt động trong sandbox, và process mới không nhận mask cũ.

> **CHỜ BỔ SUNG.** Ảnh chụp toàn bộ output `make grade` sau khi hoàn thành cả bốn task (đề yêu cầu trong bảng tự đánh giá).

8. Thảo luận và tự đánh giá

8.1. Khó khăn và cách khắc phục

Vấn đề	Nguyên nhân	Cách khắc phục
Q2 gợi ý xem `user/initcode.S` nhưng tệp không tồn tại	Bản xv6-labs-2026 chạy `/init` trực tiếp bằng `kexec()` trong `forkret()`	Đọc `forkret()` và `user/init.c`; đối chiếu `p->name = "init"`, `pid = 1` để kết luận system call đầu tiên là `open("console")`.
`si` tại lệnh `lw` làm gdb báo `target is running` và mất quyền điều khiển	Bước đơn đi vào trap trên QEMU nhiều hart	Đặt `b kerneltrap if $scause == 0xd` rồi `c` để đọc `scause`, `sepc`, `stval` ngay khi trap xảy ra.
`sandbox 32768 - sh < exec.sh` chạy sau vài lệnh khác: process con của `sh` bị page fault (`scause 0xd`) trong `malloc()`	Lỗi cố ý `LAB_SYSCALL`: `uvmalloc()` không xóa trang, nên BSS của `sh` vừa exec chứa rác, biến `freep` (`0x2010`) khác 0	Tái hiện được cả khi *không* dùng sandbox, nên đây không phải lỗi của Task 3.2. Autograder boot lại cho mỗi test; ảnh demo chạy lệnh này trước tiên.
chờ bổ sung (Task 3.3, 3.4)		

Bảng 7. Các khó khăn gặp phải trong quá trình thực hiện.

8.2. Tự đánh giá theo thang điểm

Hạng mục	Điểm tối đa	Mức tự đánh giá	Minh chứng
1. Using gdb	1.5	Xuất sắc	6/6 câu, mỗi câu có output gdb/ảnh chụp thật; `answers-syscall.txt` đạt test.
2a. `interpose` plumbing	0.5	Xuất sắc	Đủ prototype, stub, số hiệu, bảng `syscalls[]`, handler; build `-Werror` không cảnh báo; `sandbox`, `sbtest` chạy.
2b. Mask rejection	1.25	Xuất sắc	`sandbox_mask`, `sandbox_block` đạt; kiểm tra một lần trong `syscall()`; bị chặn thì trả -1, handler không chạy.
2c. Inheritance by `fork`	0.75	Xuất sắc	`sandbox_fork` đạt; `kfork()` sao chép, `freeproc()` đặt lại; demo không rò mask.
3a. Allowed pathname stored	0.75	*chờ bổ sung*	
3b. `open`/`exec` path check	1.75	*chờ bổ sung*	
4. Attack xv6	2.0	*chờ bổ sung*	
5. Chất lượng mã, git, `time.txt`	0.5	*chờ bổ sung*	Task 3.1–3.2: ba commit tách theo nội dung (trả lời gdb; plumbing; chặn và kế thừa mask).
6. Báo cáo	1.0	*chờ bổ sung*	

Bảng 8. Tự đánh giá theo rubric của đề.

9. Tài liệu tham khảo

[1] CS333 – Hệ điều hành, *Lab 2: System Calls* (đề bài, yêu cầu nộp bài và thang điểm), 2026.

[2] MIT 6.1810, *Lab: System calls*, Fall 2026. https://pdos.csail.mit.edu/6.828/2026/labs/syscall.html

[3] R. Cox, M. F. Kaashoek, R. Morris, *xv6: a simple, Unix-like teaching operating system (RISC-V)*, chương 2, chương 3 (Hình 3-3) và mục 4.3–4.4. `https://mit-pdos.github.io/xv6-riscv-book/`

[4] *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture* – thanh ghi `sstatus`, mã `scause`. `https://github.com/riscv/riscv-isa-manual`

[5] Mã nguồn xv6-labs-2026, nhánh `syscall`: `kernel/syscall.c`, `kernel/trap.c`, `kernel/trampoline.S`, `kernel/proc.c`, `kernel/vm.c`, `kernel/kalloc.c`, `kernel/memlayout.h`, `user/init.c`, `user/sandbox.c`, `user/sbtest.c`.

[6] Công cụ AI (Claude Code): hỗ trợ tự động hóa và ghi lại phiên gdb, cài đặt và kiểm thử Task 3.2, soạn thảo báo cáo. Nhóm chịu trách nhiệm về toàn bộ nội dung bài nộp. *chờ bổ sung* (khai báo AI cho Task 3.3, 3.4 nếu có).